

Software Architect Archetypes

The Painfully Precise, The Philosopher, The Approachable One, The Detailed Documenter and 8 more archetypes.



GERGELY OROSZ

JUL 11, 2023 • PAID



104



5



2

Share



What's an archetype? It's an example of a set or pattern of behaviors which are typical of a particular role. Will Larson – currently CTO at Carta – has defined four [Staff engineer archetypes](#) he's observed: Tech Lead, Architect, Solver and Right Hand.

I've been working on a new book to be published this fall, titled [The Software Engineer's Guidebook](#). While writing about the topic of software architecture, I was struck by the concept that there are complementary and contrasting software architect "archetypes," all at work alongside each other and contributing in different ways. It's a fun and potentially revealing task to identify which archetypes describe your colleagues best – maybe you'll recognize which one fits you, too!

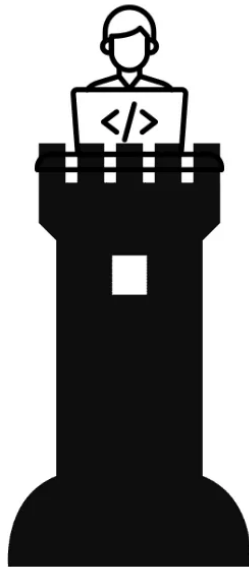
Let me know in the comments which archetype fits your approach, or if there's an archetype you've observed that's not on this list. The tech industry changes fast, and new conditions – like the rise of remote working – will likely create new archetypes.

Today, we dig into 12 software architect archetypes identified by me, except for 'The Ivory Tower Architect,' which is a common archetype in tech, and 'The Coding Machine,' which I first heard [Facebook use](#). We cover these topics:

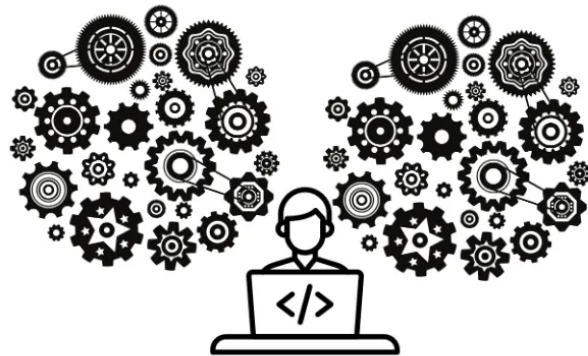
1. More theoretical archetypes
2. More practical archetypes
3. Architect or staff+ engineer?
4. Which archetypes are "better"?
5. Different archetypes pairing

Before we start, let me emphasize that these archetypes are just ways of describing behavior patterns. As we know, behaviors can change over time and also quickly. For example, on one project the person leading architecture decisions may be very practical and hands on, but in their next project be much more hands off and in a more theoretical role.

Also, people also change through time. An "Ivory Tower Architect" who's seen as hard to approach by peers may grow professionally into the archetype of an "Approachable" colleague. With that, let's jump in!



The Ivory Tower Architect



The Coding Machine

pragmaticengineer.com

Two of the twelve archetypes we'll cover

Finally, consider that the below "archetypes" are perhaps more like traits, as people can — and do! — demonstrate multiple of the below at the same time. *Thank you to reader Drew Miller for pointing this out in the comments below.*

1. More theoretical archetypes

1. The Ivory Tower Architect. A software engineer who's distant from day-to-day work, with few interactions with the software engineers responsible for implementing their architecture ideas. This person is usually hard to reach, does not seem approachable to fellow engineers. Decisions this person makes feel "top-down."

As this ivory tower dweller is unapproachable, they're often blissfully unaware when their message doesn't get through, engineers don't understand their reasoning, or when there's valid objections to their decisions.

2. The Painfully Precise. This type of engineer pays very close attention to the exact details of what others say, and simply cannot overlook any error, however small. They tend to disdain and not take seriously any engineer who fails to express themselves with technical precision, as that's just sloppy.

The issue engineers face when dealing with a Painfully Precise colleague is that discussions can get bogged down in small details and definitions, while the big picture is overlooked. Less experienced engineers who are repeatedly called out for improper use of terms may start to try and avoid their Painfully Precise colleague altogether.

3. The Theory Addict. This engineer is an avid reader of books, papers and case studies. They'll suggest a pattern or approach they read about, and cite classic or trending books, articles and case studies to justify their decisions.

A problem with The Theory Addict is they can over rely on "book smarts," but lack hands-on, practical experience in the domain in which they work. But in a setting where there's a "Practical Architect" archetype who can challenge their devotion to ideas, the Theory Addict can help the team by revealing alternative, innovative approaches. However, with no pushback, they may force impractical architecture approaches on colleagues, which end up causing a lot of pain and churn engineering time.

4. The Philosopher. Similar to the theory addict, the philosopher engineer can bring a lot of value to an architecture debate by rigorously examining an approach and positing alternatives, or opposing viewpoints.

The problem? Once a philosopher enters a discussion, it seems as if it may never end. For this archetype, exhaustive debate is the whole point. Therefore, this approach can

cause conflict with more practical archetypes who are happy with a 'good enough' decision, and just want to get on with building. *Thanks to [Ebi](#) who mentioned this archetype!*

5. The Superior Linguist. This engineer has memorized the technical language, knows all the jargon, and won't miss an opportunity to deploy the Correct Architecture Language. The quote below is something a Superior Linguist would say, especially in a group setting with plenty of junior engineers to be dazzled by mastery of terms:

"Surely idempotency is not a strict requirement, given the system is likely not even weakly consistent. I would suggest we look at durability, but not sure we have the right quorum for such discussion."

A helpful Superior Linguist can rapidly uplevel people around them, by explaining well-known – as well as lesser-known – concepts to engineers, often greatly enjoying the process. Less experienced engineers working with a helpful Superior Linguist will often rapidly grow their understanding of industry concepts, terms, and patterns.

However, there are less helpful Superior Linguists. These are the folks who tend to assume an iron correlation between competency as a software engineer, and linguistic dexterity. So they may dismiss input from colleagues who don't share their flair for speech.

The Superior Linguist and the Painfully Precise share traits, but the Painfully Precise archetype is usually less focused on technical jargon than the Superior Linguist. It's possible the Superior Linguist is also an Ivory Tower Architect, as they reject input from colleagues who didn't "eat" the dictionary like they did. This means almost every software engineer they work with can be safely ignored as inferior.

6. The Walk-Away Advisor. An experienced engineer who provides advice during the early phases of a project, but then walks away and isn't involved in the implementation phase. The problem is their advice may cause more harm than good, as assumptions made earlier during planning may need to be revised, or turn out to be wrong once work is underway.

If a team gets struck, what do they do? Stick to the walk away's earlier advice, even if it looks increasingly irrelevant? Or go back to them and seek new advice? Or simply take matters into their own hands, given the Walk-Away Advisor has no skin in the game, anyway?

Someone who fits the Walk-Away Advisor archetype could be an architect who's overstretched, with too much on their plate. They might love to be involved in projects, but simply don't have the time to commit. If you find yourself fitting the Walk-Away Advisor archetype, consider reducing the number of things you're working on, so you can see more efforts through to the end. While being a Walk-Away Advisor can *feel* like you are helping a lot; consider that walking away could be damaging for projects.

2. More practical archetypes

7. The Coding Machine. An experienced software engineer who has their head down coding for much of the time. This person tends to get things done fast, and is naturally very approachable for software engineers as they're such a familiar presence.

At Meta, "Coding Machine" was a dedicated Senior Staff Engineer archetype first created for [Michael Novati](#) – now cofounder of software engineering mentorship platform, [Formation](#). He shared with me how this archetype came about, which we cover in the article [Inside Facebook's engineering culture](#):

"Facebook takes fairness as the #1 consideration. I was basically the "largest volume" committer at the company, but when they calibrated me against the E7 in the other areas – which granted was a very small number of people, about 100 – they didn't feel like I was as impactful, or on par.

I was pushing hard to have my contributions recognized more. I did also do 300+ interviews at that time, and worked on important things on the side. My understanding is that my Director of Engineering at the time wrote up a Coding Machine archetype and presented it to the calibration group that reviews all E7s at the company. The committee accepted this, and added the archetype to the list of archetypes."

"Coding machine" is an archetype not often associated with software architects, but I think it's important to bust the myth that architects cannot be very hands-on in coding. These people exist and some of the largest tech companies – like Meta – recognize the importance of this archetype.

8. The Integrator. An experienced engineer who understands most systems a company has, how they work, what they do, and can extend or modify them with ease. These engineers are hands-on and can quickly make modifications, integrate new functionality, or to migrate from one system to the other.

The Integrator is extremely useful at companies with multiple, complex systems; be they startups, scaleups, or Big Tech. Thanks to their hands-on knowledge, integrators can offer smart workarounds and elegant "hacks" which avoid large, unwieldy, rewrite projects. *As a reminder, [engineering cultures at Big Tech can also be low-quality, as we covered](#).*

There is a danger an Integrator gets too used to patching systems here and there – after all, they're very good at it! An Integrator pairing with a more theoretical archetype could be a great way to tackle large rewrites or re-architectures, as the Integrator naturally pushes back on unnecessary rewrites.

Both Integrators and Coding Machines tend to be excellent at debugging and solving challenging bugs and outages, as they each possess a deep understanding of the code and systems, along with hands-on knowledge.

However, these archetypes tend to struggle in getting promoted to above Staff engineer level at larger companies, as promotions tend to reward more theoretical types. This is unless engineering leadership consciously rewards such archetypes like Meta did, by introducing the Coding Machine archetype to the career framework.

9. The Approachable one. An experienced software engineer who's unexpectedly approachable and available. This is often because they get involved in the work of a team by participating in discussions with engineers, going oncall, and showing up in popular chats where engineers hang out. They're also good listeners, efficient teachers

and engineers, who make you forget they hold a fancy title which few other developers at the company have.

This archetype frequently mentors less experienced engineers informally – for example, via code reviews and pairing – and sometimes more formally as well, by meeting mentees regularly to discuss challenges.

10. The Detailed Documenter. An engineer who prolifically generates documents, helping other engineers to understand architecture, common concepts and terms. You can bet this archetype is a major proponent of [RFCs](#), [design docs](#), [architecture decision records \(ADRs\)](#), runbooks, and other materials which capture and spread knowledge.

An Ivory Tower Architect may also be a Detailed Documenter, and so could an Integrator, too. This may depend on how practical or not the documentation is; a detailed documenter might live in the world of theory, or value only practical information.

11. The New and Shiny Chaser. An engineer who loves to jump to using the “latest and greatest” frameworks or approaches they can find. This approach tends to work well for architects working with less experienced teams who tend to overestimate the value of being cutting edge – and love tinkering with tools and frameworks at the bleeding edge of technology.

But over time, the approach of jumping on new tech crazes without too much consideration, may come back to bite the Chaser. Unexpected problems almost always emerge due to choosing less battle-tested approaches, or blending ideas which don't go together well.

12. The Old Schooler. The opposite of the New and Shiny Chaser, this is an experienced engineer who prefers the *same* toolset that's worked for decades, thanks very much, even though colleagues see these tools or approaches as dated and belonging in a museum.

It's possible an Old Schooler used to be a New and Shiny Chaser earlier in their career, until they got burnt enough times by novelty to decide to settle on a way of working that works for them, and to leave being innovative to others.

The Old Schooler may be popular – and advocate for a helpful approach – with more experienced teams who share their appreciation for tried-and-tested approaches, with the benefit for software development of fewer surprises. These are often engineers who swear by the rule: “choose boring technology.” They’re likely familiar with the website [BoringTechnology.club](https://boringtechnology.club), created by software engineer [Dan McKinley](#).

On this site, Dan presents a visual essay on his own journey from being a New and Shiny Chaser, to an Old Schooler who actively chooses boring technology. As Dan writes:

“If you go up to a developer and say, “hey, what would make you happy?” you often get highly specific answers. “I would be happy if I could write [Clojure](#) at work,” they’ll say, “so you should let me do my work in Clojure.” I don’t discount that when they do this, developers are genuinely describing some kind of endorphin-rich experience they have had at some point.

However, I am skeptical that the state that they’re describing is the highest level of spiritual attainment possible.

But I, too, was once like this.”

He shares how his thinking has evolved:

“If you think about innovation as a scarce resource, it starts to make less sense to also be on the front lines of innovating on databases. Or on programming paradigms.

The point isn’t that these things can’t work. Of course they can work. And there are many examples of them actually working.

But software that’s been around longer tends to need less care and feeding than software that just came out.”

Dan presents a train of thought that is worth digesting. If you consider yourself a New and Shiny Chaser, I strongly recommend [you read it](#). If you are more of an Old Schooler,

you'll probably get validation of your approach, and the comforting knowledge that you're far from alone. This thought from Dan's [site](#) is worth repeating:

"New tech typically has more known unknowns, and many more unknown unknowns. And this is really important."

Of course, the Old Schooler approach *does* create tension with team members who want to be more modern, and could make it challenging to hire engineers keen to work with more cutting-edge approaches.

3. Architect or Staff+ engineer?

The Pragmatic Engineer rarely uses the term "Architect." The reason is that Big Tech and many younger startups have dropped the title in favor of "Staff/Principal/Distinguished Engineer." These companies tend to emphasize "Engineer" to make clear that they expect experienced engineers to be pretty hands-on, and so the job title reflects this. As we mostly cover Big Tech and startups, "Staff+ engineer" is the commonly-used phrase used here.

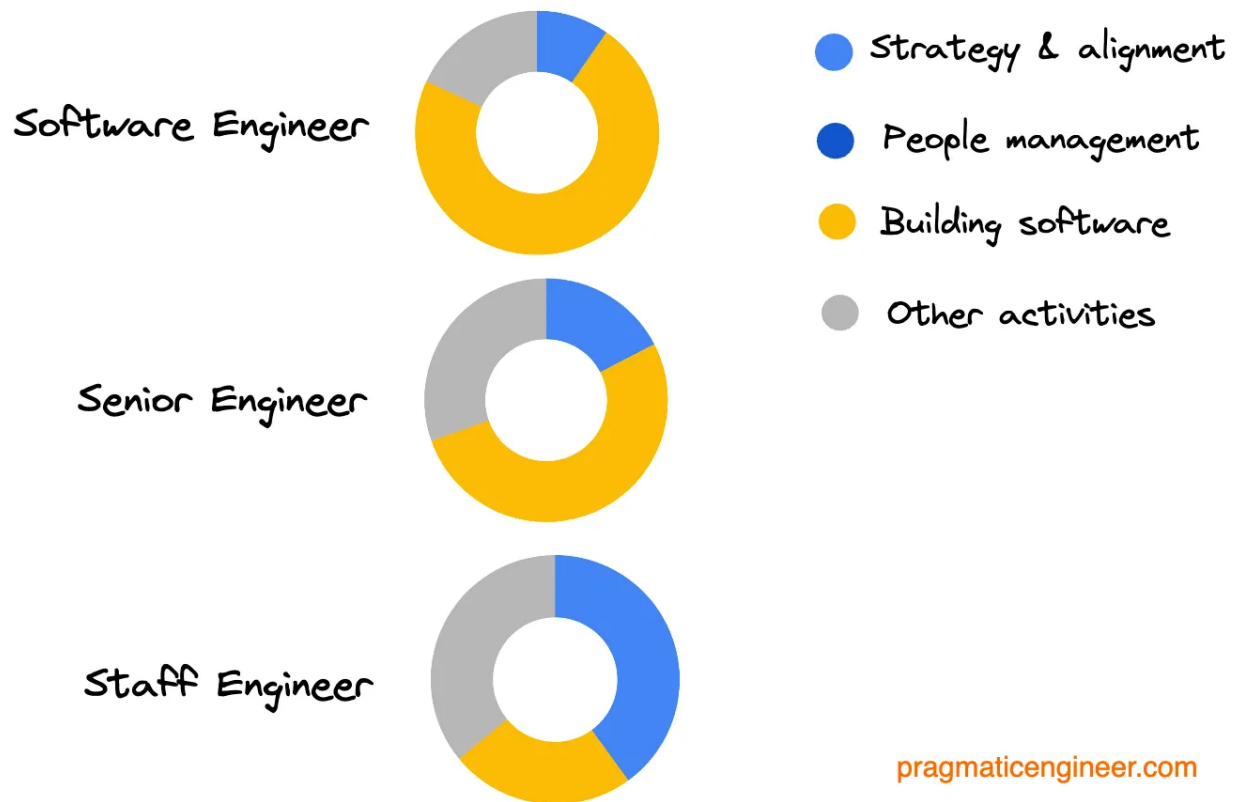
However, "Staff+ software engineer" and "Software architect" are often interchangeable. After all, Staff+ engineers are usually the most senior engineers in a team – and sometimes within a group, or org. It's only natural they tend to be heavily involved in architecture and lead many efforts. It's common to hear engineers refer to the engineer with the most input into a design as the "architect" of that project. More often than not, this is the most senior engineer, and a Staff+ engineer.

In this article, I've focused on software engineers who do a *lot* of architecting. They might not be Staff+ engineers; they could be senior engineers, or even founding engineers who grew with the business.

Also, there's plenty of companies with official roles like "Software Architect." Places which use the title "Architect" tend to be more traditional, and have usually been around for 20 years or longer. However, some companies deliberately put roles like "Architect" in place, to encourage those in the role to focus more on the business, regulatory and compliance side of things.

In the end, as engineers become more senior, they spend less time building software and more time on other things:

How do you typically spend time in each role?



pragmaticengineer.com

With seniority, engineers tend to spend less time directly building software. Source: [Engineering leadership skill set overlaps](#)

In the article [Engineering leadership skill set overlaps](#) we cover how, with seniority, most engineers spend less time building software and on things like planning, coding, code reviews and similar activities. Instead, they spend more time on strategy and alignment. And as their capacity for building shrinks, the amount of time spent on planning tends to increase. So it's only natural that most experienced engineers will have much more architecture input, and be seen as architects by their teams.

4. Which archetypes are “better?”

I *do* have a bias towards more practical archetypes, as I've repeatedly observed that engineers who stay close to the code tend to consistently make better decisions than engineers who are more distant from this core part of engineering.

However, there is no "best" or "worst" approach when it comes to architecture. More theoretical archetypes can be very useful: for example, in projects with a major blast radius. Also, when there are hard-to-revert decisions to make, more theoretical archetypes can offer organized approaches to planning and conduct detailed planning, which is exactly what the situation needs.

Also, more theoretical archetypes often spend more time on understanding the business and the industry. This can bring much-needed insight when deciding *the right thing* to build.

Meanwhile, practical archetypes bring the benefit of having a bias to action, and of jumping straight into building and coding. They are hands-on with codebases, and tend to move faster than most senior engineers in implementing their ideas.

Different stages of a team and a company call for different archetypes in order to reach their goals:

- **During the early stages of a product:** practical archetypes will be much more helpful. Moving fast and shipping quickly are more important than doing detailed planning. After all, it's not even clear if customers will find the product useful! Rapid iteration can be done with more practical archetypes.
- **During a product's growth and scaling up:** a mix of practical and theoretical archetypes can be helpful. This is where it's more important to understand the trajectory of the business, often by spending time with product folks, data scientists, business people and the leadership. All this is time spent away from the keyboard, building up mental models in your mind to better understand the business.
- **Maintaining a mature product** can be done by both theoretical and practical archetypes. By this time, there's usually less urgency to development.

- **Operating in a highly regulated environment** tends to fit theoretical archetypes well. This is because it's more important to play safe and triple check the business impact of all changes.
- **Maintaining legacy systems** tends to push engineers to become more theoretical archetypes, as the work is more about understanding and not breaking anything, and less about building.
- **"Wartime"** tends to reward more practical archetypes who take the lead. This is because moving and building quickly is much more important than building consensus. Taking a long time to make decisions is less tolerated.
- **"Peacetime"** is when more theoretical archetypes tend to thrive. There's time and appetite to do things properly, and not messing things up is often more important than moving quickly. *We cover more on this in ["Peacetime" vs "wartime" at tech companies](#).*

5. Different archetypes pairing

For proof that there are no "good" or "bad" archetypes, I think of the single best architect I've had the opportunity to work with. This wasn't one person, but a pair: a highly theoretical architect archetype, and a very practical one.

These two kept challenging each other throughout their collaboration. The unique pairing led to practical architecture that was built with long-term maintainability and extensibility, all in an elegant way. Each engineer developed more respect for the other's way of working: the theoretical archetype started to become more practical, and the overly practical archetype started to appreciate the value of research papers, books and taking time to *think*, not just to code.

One of the most prolific "architect pairs" of all, is Jeff Dean and Sanjay Ghemawat at Google. Both were early engineers there, and paired up to make its search engine scaleable, by taking the original prototype and making it handle the exponential traffic Google started to receive as it grew. They also built MapReduce, a core system for processing large-scale data. Jeff and Sanjay were the first and only people to be promoted to Google Senior Fellows – and were the company's first Level 11 engineers.

Today, Jeff Dean leads Google's AI efforts, and Sanjay Ghemawat remains Google's most prolific individual contributor.

In 2018, The New Yorker published an in-depth profile of the pair titled "[The friendship that made Google huge](#)." Two interesting things stand out in this piece.

First, Jeff and Sanjay wrote most of their code by pair programming at a single computer. And they were remarkably efficient pair programmers. From the New Yorker article:

"Although developers sometimes talk about 'pair programming' — two programmers sharing a single computer, one 'driving' and the other 'navigating' — they usually conceive of such partnerships in terms of redundancy, as though the pair were co-pilots on the same flight. Jeff and Sanjay, by contrast, sometimes seem to be two halves of a single mind. Some of their best-known papers have as many as a dozen co-authors. Still, Bill Coughran, one of their managers, recalled, 'They were so prolific and so effective working as a pair that we often built teams around them.'

I agree with the observation that it's very rare to see such an intense and long-term pairing between engineers. I've seen plenty of engineers give pairing a go and learn a lot from it, but I've yet to observe pairings of engineers that last years.

The other thing which struck me in this in-depth profile, was how *different* Jeff and Sanjay are; they're contrasting personalities. If we were to use archetypes as an analogy, they were clearly different types. From [the New Yorker article](#):

"Jeff is great at coming up with wild new ideas and prototyping things," Wilson Hsieh, their longtime colleague, said. "Sanjay was the one who built things to last." In life, Jeff is more outgoing, Sanjay more introverted. In code, it's the reverse. Jeff's programming is dazzling—he can quickly outline startling ideas—but, because it's done quickly, in a spirit of discovery, it can leave readers behind. Sanjay's code is social.

"Some people," Silverstein said, "their code's too loose. One screen of code has very little information on it. You're always scrolling back and forth to figure out what's going on." Others write code that's too dense: "You look at it, you're, like,

'Ugh. I'm not looking forward to reading this.' Sanjay has somehow split the middle. You look at his code and you're, like, 'O.K., I can figure this out,' and, still, you get a lot on a single page. Whenever I want to add new functionality to Sanjay's code, it seems like the hooks are already there. I feel like Salieri [classical music composer and peer of Mozart.] I understand the greatness. I don't understand how it's done."

Pairing different personalities and archetypes can create wonderful outcomes, as Jeff Dean and Sanjay Ghemawat show. However, it is *rare* to see pairing work out so well; at least I've seen it only a few times during my career, to date.

Then again, I've seen relatively few people do proper pair programming. So one way to increase the chances of finding a pairing partner with whom you can become much more productive, is to just give pair programming a go. Try pairing with different engineers on your team and see what happens!

Takeaways

Software engineer archetypes is a fun topic, especially for identifying which ones fit current or past colleagues. But there is one benefit beyond that: as an aid to self-reflection.

You can bucket people on your team to various archetypes, but how would your colleagues "bucket you," based on how you architect things? Are you theoretical, or more practical?

Thinking about archetypes can make you consider how you *consciously* approach software engineering. Are you a Philosopher? An Integrator? A New And Shiny Chaser? Most importantly: are you the archetype you wish to be?

Most engineers will recognize themselves as a mix of archetypes, the combination of which changes based on the project and team dynamics. Across a long-enough career, you'll likely find yourself operating as each archetype at different times. Knowing these archetypes can help you be mindful about what you do, and to decide if a role is right for you.

Don't forget that these archetypes describe behaviors, and that behaviors can, and do, change. No matter which archetype you are today, tomorrow you could change your behavior and approach, and become something different.

Use these archetypes to reflect on where you are now. If you're unhappy about being the archetype you are today, then make changes to be the one which best suits you and the context you're working in.

Do you recognize yourself – or your former self – in any of the archetypes? Feel free to add a comment. The workplace is changing fast and the rise of remote working is likely to create new archetypes for new times. Do you know about any archetype you've observed at work, which isn't on this list? Share it below:

Thanks to [Ebi Atawodi](#), [Frank de Jonge](#) and [Willem Spruijt](#) for their input and review of this article.



104 Likes · 2 Restacks

5 Comments



Write a comment...



Drew Miller Writes I regret everything Jul 11 ❤️ Liked by Gergely Orosz

Great article - I think some of these characteristics are more like "traits" than archetypes, as I've seen people demonstrate multiple of them at once.

♡ LIKE (16) 💬 REPLY ...

1 reply by Gergely Orosz



Matt D. Jul 15 ❤️ Liked by Gergely Orosz

Wow, this article has some great timing for mid year performance reviews!

I was recently informed that I was a very approachable, honest and flexible Software Engineer that could help many others across the company reach their goals. Along with that I can see how I fit into some other archetypes that may not be so beneficial in some situations.

I'll also be using this article's wisdom for mentoring a few Staff+ Engineers on their work as I can already see how each one has a different archetype listed in the article. Being able to see this archetype pattern will make me a better mentor to these Staff+ Engineers and bring a lot of value to the company.

Thank you again for taking the time to write this article!

♡ LIKE (1) 💬 REPLY ...

3 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing